CCE 2 Unit 5 Report

1. Project Title: Develop emergency centers and connecting roads as a weighted graph, and apply BFS or DFS to analyze connectivity and Dijkstra's algorithm to find the shortest route to an emergency location**.**

2. Problem Statement:

In emergency situations such as road accidents, fires, and medical emergencies, reaching the destination quickly is essential to save lives and reduce damage. However, finding the shortest route between emergency centers can be difficult when multiple roads and alternative paths are available.

The objective of this project is to develop a C++ program that represents emergency centers and their connecting roads using a weighted graph. In this graph, each vertex represents an emergency center, while each edge represents a road connecting two centers. The weight of each edge represents the distance between the centers in kilometers.

The system allows users to enter the number of emergency centers, the number of connecting roads, the road connections, and their respective distances. Users can also specify a source center and a destination center according to their requirements.

Breadth-First Search (BFS) is used to traverse the graph and analyze the connectivity of emergency centers. Dijkstra's algorithm is used to calculate the shortest distance and identify the most efficient route from the source center to the destination center.

Finally, the program displays the BFS traversal, the shortest route, and the total distance required to reach the destination. If no route exists between the selected centers, the program displays an appropriate message.

This project demonstrates the practical application of graph data structures and graph traversal algorithms in emergency route planning. It helps users understand network connectivity and shortest-path calculations, which can support efficient route selection for ambulances, fire brigades, and other emergency services.

3. **Objectives:**

1. To represent emergency centers and connecting roads using a weighted graph.
2. To implement Breadth-First Search (BFS) for traversing the graph and analyzing connectivity between emergency centers.
3. To implement Dijkstra's algorithm to find the shortest route between a source and destination.
4. To calculate the minimum distance required to reach an emergency location.
5. To accept user input for emergency centers, road connections, distances, source, and destination.
6. To identify whether a route exists between the selected emergency centers.
7. To display the BFS traversal, shortest route, and total distance in a clear format.
8. To understand the practical application of graph data structures and algorithms in emergency route planning.

4. **Theory:**

A graph is a non-linear data structure consisting of vertices and edges. In this project, emergency centers are represented as vertices, and the roads connecting these centers are represented as edges. Each edge has a weight that represents the distance between two emergency centers in kilometers. This type of graph is called a **weighted graph**.

The main purpose of this project is to analyze connectivity between emergency centers and find the shortest route from a source center to a destination center using graph algorithms. The program is implemented in C++ and accepts user input for the number of centers, connecting roads, road distances, source, and destination.

1. Weighted Graph:

A weighted graph represents a network in which each edge has a numerical value called a weight. In this project, the weight represents the distance between two emergency centers. The graph helps represent road connections and provides the basic structure for route analysis.

2. Breadth-First Search (BFS):

Breadth-First Search is a graph traversal algorithm that visits vertices level by level. It starts from a selected source vertex and visits all its adjacent vertices before moving to the next level. A queue is used to maintain the order of traversal, and a visited array prevents vertices from being visited repeatedly. BFS helps determine which emergency centers are reachable from the selected source.

3. Dijkstra's Algorithm:

Dijkstra's algorithm is a shortest-path algorithm used to find the minimum distance between a source vertex and other vertices in a weighted graph. It calculates the shortest distance by selecting the unprocessed vertex with the smallest known distance and updating the distances of its neighboring vertices. A priority queue can be used to improve efficiency. The algorithm also stores parent vertices to reconstruct the shortest route.

Dijkstra's algorithm works correctly when all edge weights are non-negative.

5.Algorithm:

Step 1: Start the program.

Step 2: Enter the number of emergency centers (vertices).

Step 3: Enter the number of connecting roads (edges).

Step 4: Enter the road connections between centers and their distances in kilometers.

Step 5: Represent the emergency centers and roads using a weighted graph.

Step 6: Enter the source emergency center and destination emergency center.

Step 7: Apply Breadth-First Search (BFS) to traverse the graph from the source center and display all reachable centers.

Step 8: Initialize the distance of the source center to zero and the distances of all other centers to infinity.

Step 9: Apply Dijkstra's algorithm to calculate the shortest distance from the source to the destination.

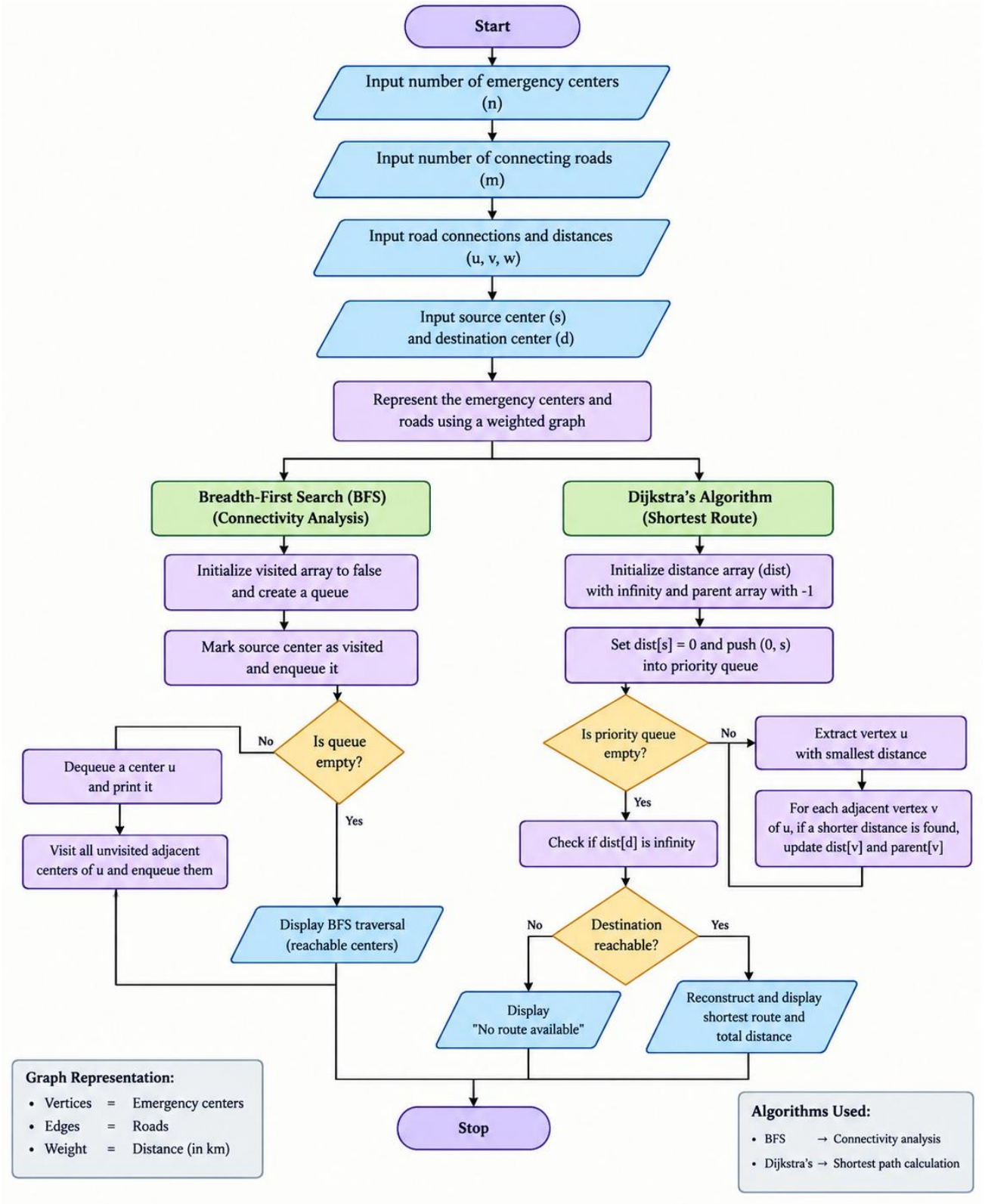
Step 10: Update the distances of neighboring centers whenever a shorter route is found.

Step 11: Store the parent of each center to reconstruct the shortest route.

Step 12: If the destination is reachable, display the shortest route and total distance. Otherwise, display a message indicating that no route is available.

Step 13: Stop the program.

6. Flowchart:


Start
Input number of emergency centers (n)
Input number of connecting roads (m)
Input road connections and distances (u, v, w)
Input source center (s) and destination center (d)
Represent the emergency centers and roads using a weighted graph
Breadth-First Search (BFS) (Connectivity Analysis)
Initialize visited array to false and create a queue
Mark source center as visited and enqueue it
Is queue empty?
No
Yes
Dequeue a center u and print it
Visit all unvisited adjacent centers of u and enqueue them
Display BFS traversal (reachable centers)
Dijkstra's Algorithm (Shortest Route)
Initialize distance array (dist) with infinity and parent array with -1
Set dist[s] = 0 and push (0, s) into priority queue
Is priority queue empty?
No
Yes
Extract vertex u with smallest distance
For each adjacent vertex v of u, if a shorter distance is found, update dist[v] and parent[v]
Check if dist[d] is infinity
Destination reachable?
No
Yes
Display "No route available"
Reconstruct and display shortest route and total distance
Stop
Graph Representation:
Vertices = Emergency centers
Edges = Roads
Weight = Distance (in km)
Algorithms Used:
BFS → Connectivity analysis
Dijkstra's → Shortest path calculation

7. Code:

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
using namespace std;

void   BFS(vector<pair<int,int>>   g[],
int n, int s) {
   vector<bool> visited(n + 1, false);
   queue<int> q;
   q.push(s);
   visited[s] = true;

   cout << "BFS Traversal: ";
   while (!q.empty()) {
      int u = q.front();
      q.pop();
      cout << u << " ";

      for (auto e : g[u]) {
         if (!visited[e.first]) {
            visited[e.first] = true;
            q.push(e.first);
         }
      }
   }
   cout << endl;
}

void      dijkstra(vector<pair<int,int>>
g[], int n, int s, int d) {
```

```
    vector<int> dist(n + 1, INT_MAX),
parent(n + 1, -1);
    priority_queue<pair<int,int>,
        vector<pair<int,int>>,
        greater<pair<int,int>>> pq;

    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        int u = pq.top().second;
        pq.pop();

        for (auto e : g[u]) {
            int v = e.first, w = e.second;

            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                parent[v] = u;
                pq.push({dist[v], v});
            }
        }
    }

    if (dist[d] == INT_MAX) {
        cout << "No route available!\n";
        return;
    }

    cout << "Shortest Distance: " <<
dist[d] << " km\n";
    cout << "Shortest Route: ";
```

```
    vector<int> path;
    for (int v = d; v != -1; v = parent[v])
        path.push_back(v);

    for (int i = path.size() - 1; i >= 0; i--
) {
        cout << path[i];
        if (i > 0) cout << " -> ";
    }
    cout << endl;
}

int main() {
    int n, m, u, v, w, s, d;

    cout << "Enter number of
emergency centers: ";
    cin >> n;

    vector<pair<int,int>> g[n + 1];

    cout << "Enter number of roads: ";
    cin >> m;

    cout << "Enter roads (center1
center2 distance):\n";
    for (int i = 0; i < m; i++) {
        cin >> u >> v >> w;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    cout << "Enter source center: ";
```

```
    cin >> s;
    cout << "Enter destination center: ";
    cin >> d;

    BFS(g, n, s);
    dijkstra(g, n, s, d);

    return 0;
}
```

8. Difference Between Array and Linked List:

Sr. No.	BFS	Dijkstra's Algorithm
1.	BFS stands for Breadth-First Search.	Dijkstra's algorithm is a shortest-path algorithm.
2.	It is used for graph traversal.	It is used to find the shortest path.
3.	It uses a Queue.	It commonly uses a Priority Queue.
4.	It visits vertices level by level.	It selects the vertex with the minimum known distance.
5.	It does not consider edge weights when traversing.	It considers edge weights to calculate the shortest distance.
6.	It finds the shortest path in an unweighted graph.	It finds the shortest path in a weighted graph with non-negative edge weights.
7.	It is useful for checking connectivity.	It is useful for finding the minimum-distance route.

9. Result:

The Emergency Route Analysis system was successfully implemented using C++ programming and graph data structures. The emergency centers were represented as vertices, and the roads connecting them were represented as weighted edges, where each weight indicated the distance between two centers in kilometers.

The program accepted user input for the number of emergency centers, connecting roads, road distances, source center, and destination center. Breadth-First Search (BFS) was successfully applied to traverse the graph and identify reachable emergency centers from the selected source.

Dijkstra's algorithm was implemented to calculate the shortest distance between the source and destination. The program displayed the shortest route, the total distance in kilometers, and an appropriate message if no route was available.

The results demonstrated that BFS is useful for analyzing connectivity, while Dijkstra's algorithm efficiently determines the minimum-distance route in a weighted graph with non-negative edge weights. Thus, the project successfully achieved its objectives and demonstrated the practical application of graph traversal and shortest-path algorithms in emergency route planning. It also improved understanding of weighted graphs, queue-based traversal, priority queues, and shortest-path calculation in real-world situations.

10. Conclusion :

The Emergency Route Analysis project was successfully developed using C++ programming and graph algorithms. The system represents emergency centers and connecting roads as a weighted graph, where vertices represent emergency centers and edges represent roads with their respective distances.

Breadth-First Search (BFS) was used to analyze connectivity between emergency centers, while Dijkstra's algorithm was used to determine the shortest route and minimum distance between a source and destination. The program accepts user input and displays the BFS traversal, shortest route, and total distance.

This project provides a clear understanding of graph data structures and their practical applications in route planning. It can be useful in emergency services such as ambulance routing, fire brigade operations, and hospital transportation.

Thus, the project demonstrates how graph algorithms can help identify efficient routes and support faster decision-making during emergency situations.